

**Universidad Nacional de Cuyo**  
**Facultad de Ingeniería**

Microcontroladores y Electrónica de Potencia

# **Trabajo Integrador: Extrusora de PET para Filamento 3D**

Alumno: Nicolás Sorrentino  
Legajo: 13423

Mendoza, Febrero 2026

# Índice

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                             | <b>3</b>  |
| 1.1. Motivación y Contexto . . . . .                               | 3         |
| 1.2. Fundamentos y Principios de Funcionamiento . . . . .          | 3         |
| <b>2. Esquema Tecnológico</b>                                      | <b>3</b>  |
| <b>3. Detalle de Módulos</b>                                       | <b>4</b>  |
| 3.1. Microcontrolador: STM32F103 (Blue Pill) . . . . .             | 4         |
| 3.2. Sistema de Alimentación y Regulación . . . . .                | 5         |
| 3.3. Control de Tracción: Motor Paso a Paso y Driver . . . . .     | 5         |
| 3.4. Control de Temperatura: Sensado y Actuación Térmica . . . . . | 6         |
| 3.4.1. Sensor de Temperatura (NTC 100k $\Omega$ ) . . . . .        | 6         |
| 3.4.2. Actuador Térmico (Cartucho Calefactor) . . . . .            | 6         |
| 3.5. Interfaz de Usuario Local . . . . .                           | 7         |
| 3.5.1. LCD 1602 I2C . . . . .                                      | 7         |
| 3.5.2. Encoder Rotativo con Pulsador . . . . .                     | 8         |
| 3.6. Sistema de Señalización . . . . .                             | 8         |
| 3.6.1. LED RGB de Estado . . . . .                                 | 8         |
| 3.6.2. Buzzer Activo . . . . .                                     | 9         |
| 3.7. Botones de Control Físico . . . . .                           | 9         |
| 3.8. Comunicación Serial UART . . . . .                            | 9         |
| <b>4. Funcionamiento General</b>                                   | <b>10</b> |
| 4.1. Arquitectura del Firmware . . . . .                           | 10        |
| 4.1.1. Timers del Sistema . . . . .                                | 10        |
| 4.1.2. Adquisición de Temperatura (ADC + DMA) . . . . .            | 11        |
| 4.1.3. Bucle Principal (Superloop) . . . . .                       | 11        |
| 4.2. Máquina de Estados Térmica . . . . .                          | 11        |
| 4.3. Máquina de Estados del Motor . . . . .                        | 12        |
| 4.4. Sistema de Seguridad Multicapa . . . . .                      | 13        |
| 4.4.1. Detección de Errores Críticos . . . . .                     | 13        |
| 4.4.2. Enclavamiento de Seguridad (Safety Latch) . . . . .         | 13        |
| 4.4.3. Watchdog Independiente (IWDG) . . . . .                     | 13        |
| 4.5. Interacción de Módulos y Flujo de Ejecución . . . . .         | 13        |
| 4.6. Diagrama de Flujo Operativo . . . . .                         | 14        |
| <b>5. Programación</b>                                             | <b>15</b> |
| 5.1. Configuración de Periféricos . . . . .                        | 15        |
| 5.2. Descripción del Firmware . . . . .                            | 15        |
| <b>6. Etapas de Montaje y Ensayos Realizados</b>                   | <b>15</b> |
| <b>7. Resultados y Especificaciones Finales</b>                    | <b>15</b> |
| <b>8. Conclusiones</b>                                             | <b>15</b> |
| <b>9. Referencias</b>                                              | <b>15</b> |



# 1. Introducción

El presente trabajo describe el desarrollo de una celda de extrusión automatizada para el procesamiento de fibras de polietileno tereftalato (PET) reciclado. Este proyecto se integra en el marco de las Becas de Estímulo a las Vocaciones Científicas (EVC-CIN) y tiene como objetivo técnico la producción de filamento calibrado para su uso en manufactura aditiva, promoviendo soluciones de economía circular y reutilización de polímeros de ingeniería.

## 1.1. Motivación y Contexto

La gestión de residuos plásticos representa uno de los desafíos ambientales más críticos de la actualidad. El PET posee propiedades mecánicas que permiten su revalorización mediante procesos de termo-formado. Este desarrollo busca transformar tiras de botellas en un insumo tecnológico (filamento), facilitando investigaciones sobre nuevos materiales como el refuerzo de matrices cementicias.

## 1.2. Fundamentos y Principios de Funcionamiento

El sistema opera bajo el mando centralizado de un microcontrolador **STM32F103**, que actúa como el núcleo de procesamiento encargado de coordinar todas las variables del proceso. El funcionamiento se basa en la plastificación térmica del polímero y su tracción mecánica constante.

El ecosistema de hardware se fundamenta en los siguientes pilares:

- **Gestión Centralizada:** El microcontrolador supervisa en tiempo real una máquina de estados que integra la interfaz de usuario (HMI), la seguridad y los algoritmos de control.
- **Conversión Térmica:** Se utiliza un cartucho calefactor de 12V/40W y un sensor NTC 100k, gestionados por un lazo cerrado con algoritmo **PID**.
- **Sistema de Tracción:** La fuerza es proporcionada por un motor paso a paso con driver **DRV8825**, configurado a 1/32 micropasos para asegurar la uniformidad del diámetro.
- **Interfaz HMI:** Visualización en LCD 1602 I2C y control mediante encoder rotativo.

# 2. Esquema Tecnológico

El siguiente diagrama describe la interacción entre los módulos bajo la gestión del STM32F103.

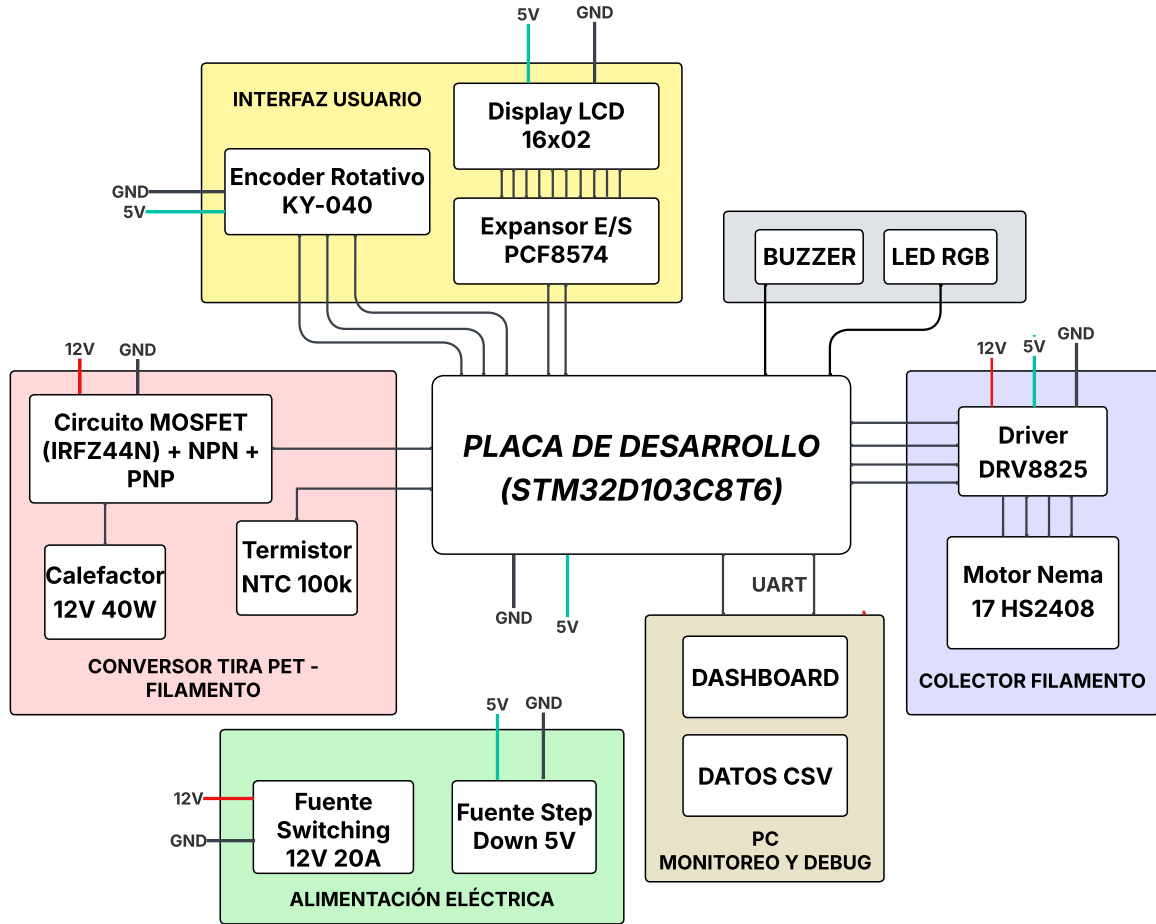


Figura 1: Diagrama de bloques funcional de la extrusora de PET.

### 3. Detalle de Módulos

*Nota: Los pines del microcontrolador mencionados en esta sección corresponden a la asignación configurada en STM32CubeMX. La configuración específica de cada periférico se detalla en la Sección 5 (Programación).*

#### 3.1. Microcontrolador: STM32F103 (Blue Pill)

El núcleo del sistema es el microcontrolador **STM32F103C8T6**, basado en arquitectura ARM Cortex-M3 de 32 bits operando a 72 MHz.

- **Conversión ADC con DMA:** Adquisición continua de la señal del termistor sin consumir ciclos de CPU.
- **Timers:** Generación de señales PWM para el calefactor y pulsos de paso para el motor.
- **Periféricos de comunicación:** I2C para el LCD, UART para telemetría PC y GPIO para botones/encoder.

- **Memoria Flash:** 64 KB, de los cuales se reserva una página (1 KB) para almacenamiento no volátil de parámetros PID y setpoints.

### 3.2. Sistema de Alimentación y Regulación

El sistema es alimentado por una fuente conmutada de **12V/20A**, suficiente para manejar las cargas combinadas del cartucho calefactor (40W) y el motor paso a paso. La distribución de potencia se realiza de la siguiente forma:

- **Bus de 12V:** Alimenta directamente el cartucho calefactor (vía MOSFET), el driver DRV8825 y el regulador step-down.
- **Bus de 5V (LM2596):** Regulador conmutado que alimenta la Bluepill, el LCD I2C, encoder rotativo, LED RGB y buzzer.
- **Bus de 3.3V:** Suministrado por el regulador interno del STM32 (AMS1117 de la placa Blue Pill) para la lógica digital.

### 3.3. Control de Tracción: Motor Paso a Paso y Driver

El avance del filamento se realiza mediante un motor paso a paso **NEMA 17 modelo 17HS2408** con las siguientes características:

- Ángulo de paso: 1.8° (200 pasos/revolución)
- Corriente nominal: 0.6A por bobina
- Torque de retención: 0.4 Nm

El control del motor se efectúa mediante el driver **DRV8825**, configurado en modo de **1/32 micropasos**, resultando en 6400 pasos por revolución. Esta subdivisión garantiza un movimiento suave y reduce las vibraciones mecánicas que afectarían la uniformidad del filamento.

**Pines del STM32 utilizados:**

- PA8 (TIM1\_CH1): Señal STEP (pulsos de paso generados por Timer)
- PA1: Señal DIR (dirección de giro)
- PA5: Señal ENABLE (habilitación del driver)
- PA11: Señal FAULT (detección de fallos del driver)

**Relación de Transmisión:** El motor se acopla a un reductor de engranajes de relación de transmisión 60:1, multiplicando el torque disponible.

**Forma de manejo:** El control se realiza mediante tres señales digitales. La señal STEP es generada automáticamente por un Timer configurado en modo PWM, donde la frecuencia del tren de pulsos determina la velocidad de giro. La señal DIR (nivel alto/bajo) establece el sentido de rotación, y ENABLE activa o desactiva el driver. La señal FAULT permite detectar condiciones de error en el driver (sobrecalentamiento, cortocircuito).

### 3.4. Control de Temperatura: Sensado y Actuación Térmica

El sistema térmico consta de dos elementos integrados en un bloque calefactor de aluminio tipo **E3D V6**:

#### 3.4.1. Sensor de Temperatura (NTC 100k $\Omega$ )

- Termistor negativo encapsulado en tubo metálico de 3mm de diámetro.
- Rango de operación: -50°C a 300°C.
- Conexión: Divisor resistivo con  $R_{serie} = 1k\Omega$  conectado a PA0 (ADC1\_CH0).
- El firmware aplica corrección mediante **ecuación Beta del termistor** con dos constantes Beta (3950 para baja temperatura, 4410 para alta temperatura) para compensar la no linealidad en el rango 200-250°C.

Figura 2: Circuito de acondicionamiento de señal para el termistor NTC.

#### 3.4.2. Actuador Térmico (Cartucho Calefactor)

Para la gestión térmica se utiliza una resistencia de 12V/40W (aprox. 3,6 $\Omega$ ) controlada mediante una etapa de potencia.

Se diseñó un circuito basado en un transistor MOSFET de canal N **IRFZ44N**, configurado como interruptor de estado sólido. Esta etapa permite al microcontrolador, mediante una señal PWM de 3.3V en el pin PB10, conmutar la carga de 12V necesarios para el cartucho.

Debido a que el IRFZ44N no es un dispositivo de nivel lógico, se implementó un driver de gate de 12V (push-pull con transistores **BC327** y **BC337**) para asegurar la completa saturación del componente. Esto evita que el transistor trabaje en su zona lineal, lo que provocaría un sobrecalentamiento destructivo por  $R_{DS(on)}$ .

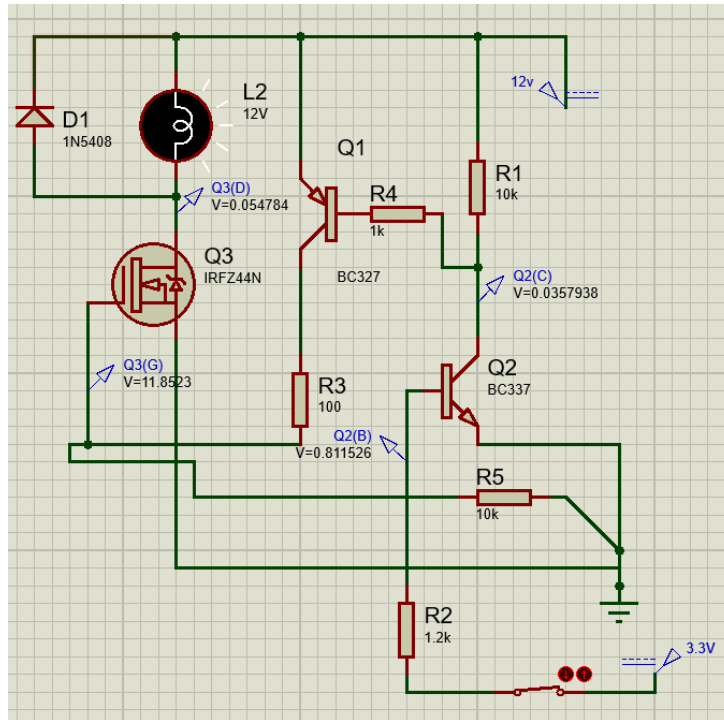


Figura 3: Circuito de control de potencia mediante MOSFET para el cartucho calefactor.

**Forma de manejo:** La lectura del termistor se realiza mediante el conversor ADC en modo DMA, que captura 64 muestras de forma autónoma sin intervención del procesador. El firmware promedia estas muestras, calcula la resistencia del termistor mediante la ley de Ohm en el divisor resistivo, y aplica la **ecuación Beta** para obtener la temperatura en grados Celsius. El control del calefactor se efectúa mediante una señal digital ON/OFF en el pin PB10, que activa la etapa de driver la cual conmuta el MOSFET de potencia según la salida del controlador PID.

### 3.5. Interfaz de Usuario Local

#### 3.5.1. LCD 1602 I2C

Para la visualización de parámetros y menús se utiliza un display alfanumérico de **16 columnas × 2 filas** con módulo adaptador I2C basado en el chip **PCF8574**. Este conversor reduce los 8 pines de datos paralelos del LCD a solo 2 líneas (SDA y SCL), simplificando el cableado.

**Características:**

- Dirección I2C: 0x27 (7 bits) / 0x4E (formato 8 bits de la HAL)
- Pines del STM32: PB6 (I2C1\_SCL) y PB7 (I2C1\_SDA)

El LCD muestra en tiempo real: temperatura actual/objetivo, estado térmico (calentando/listo), velocidad del motor en Hz y mm/s, y mensajes de alarma en caso de fallo.

**Forma de manejo:** La comunicación con el LCD se realiza mediante el protocolo I2C utilizando las funciones de la biblioteca HAL. El PCF8574 actúa como expensor de puertos, recibiendo bytes de comando y datos que son traducidos a las señales paralelas que requiere el controlador HD44780 del LCD.

### 3.5.2. Encoder Rotativo con Pulsador

La navegación por el menú de configuración se realiza mediante un **encoder rotativo incremental de 20 pulsos por revolución** con pulsador integrado.

**Conexión al STM32:**

- PB13: Señal A (CLK)
- PB14: Señal B (DT)
- PB15: Pulsador (SW)

**Forma de manejo:** El encoder genera dos señales cuadradas desfasadas  $90^\circ$  (A y B). La lectura del sentido de giro se determina comparando el estado de la señal B cuando ocurre un flanco en A: si B está en alto, el giro es horario; si está en bajo, es antihorario. El pulsador se lee como un botón estándar con pull-up. El firmware mide el tiempo de pulsación para discriminar entre acción corta (confirmación) y larga (cambio de modo).

## 3.6. Sistema de Señalización

### 3.6.1. LED RGB de Estado

Se utiliza un LED RGB de cátodo común para indicar visualmente el estado operativo del sistema:

**Código de colores implementado:**

- **Amarillo:** Sistema en reposo (Idle), listo para iniciar.
- **Azul:** Calentamiento inicial ( $T < \text{Target} - 20^\circ\text{C}$ ).
- **Cyan:** Aproximación térmica ( $T > \text{Target} - 20^\circ\text{C}$ ).
- **Verde:** Temperatura estable en rango  $\pm 5^\circ\text{C}$ , listo para extruir.
- **Rojo parpadeante (500ms):** Alarma de seguridad activa.

**Pines del STM32:**

- PA3: Canal Rojo
- PB0: Canal Verde
- PB1: Canal Azul

**Forma de manejo:** Cada canal del LED se controla mediante una salida GPIO configurada en modo push-pull. Al ser un LED de cátodo común, se requiere nivel alto (3.3V) para encender cada color. Las combinaciones de colores se logran activando dos o tres canales simultáneamente.

### 3.6.2. Buzzer Activo

Zumbador piezoeléctrico de 5V para retroalimentación auditiva:

- **Beeps cortos (50-100ms):** Confirmación de pulsación de botón o cambio de parámetro.
- **Beeps intermitentes (500ms):** Alarma de error crítico (desconexión de sensor, sobretemperatura, timeout de calentamiento). Sincronizado con el parpadeo del LED rojo.
- Pin de control: PA2

**Forma de manejo:** El buzzer se activa mediante una salida GPIO simple. Al ser un modelo activo, solo requiere alimentación (nivel alto) para generar el tono. El firmware controla la duración de los beeps mediante temporizadores no bloqueantes basados en `HAL_GetTick()`.

## 3.7. Botones de Control Físico

El sistema incorpora tres pulsadores para el control directo del proceso:

- PA4 (`START_BTN`): Inicia el motor de tracción (solo si la temperatura es segura  $> 100^{\circ}C$ ).
- PB12 (`STOP_BTN`): Detiene el motor.
- PB11 (`HEATER_BTN`): Conmuta el calentador entre ON/OFF (modo toggle).

Todos los botones implementan lógica pull-up interna y **filtrado temporal (200-300ms)** para evitar lecturas erróneas causadas por las vibraciones mecánicas que ocurren al presionar y soltar el contacto.

**Forma de manejo:** Los botones se leen mediante entradas GPIO configuradas con resistencias pull-up internas. El estado de reposo es nivel alto (3.3V), y al presionar se detecta nivel bajo (GND). Para evitar múltiples detecciones causadas por el rebote mecánico del contacto, el firmware ignora cualquier cambio de estado durante un período de 200-300ms después de la primera detección válida. Esto asegura que solo se registre una pulsación por cada acción real del usuario.

## 3.8. Comunicación Serial UART

El sistema integra un canal de comunicación bidireccional vía **UART** para supervisión remota y ajuste de parámetros desde PC.

**Configuración:**

- Periférico: USART1
- Baudrate: 115200 bps, 8N1 (8 bits de datos, sin paridad, 1 bit de stop)
- Pines: PA9 (TX) y PA10 (RX)

**Protocolo implementado:**

- **Telemetría (100ms):** Envío continuo de datos en formato CSV: timestamp, temperatura actual, target, PWM, velocidad, error PID, código de error y flags de estado.
- **Comandos ASCII:** Recepción de instrucciones para cambio de setpoint (T250), encendido/apagado de calefactor (H1/H0), arranque/parada de motor (R/S), ajuste de PID (P/I/D), guardado en Flash (W) y reset a valores de fábrica (X).

La aplicación de PC (Python/Tkinter) visualiza gráficos en tiempo real, permite control remoto y registra automáticamente logs de telemetría y eventos de seguridad en archivos CSV.

## 4. Funcionamiento General

### 4.1. Arquitectura del Firmware

El firmware implementa una arquitectura de **Superloop** (bucle principal infinito) con sincronización temporal precisa mediante interrupciones de Timer. Esta estrategia es común en sistemas embebidos de tiempo real, ya que permite respuestas deterministas sin la sobrecarga de un RTOS.

#### 4.1.1. Timers del Sistema

El sistema utiliza dos periféricos timer con funciones específicas:

##### **TIM1 (Advanced Timer) - Generador PWM para Motor:**

- Opera a 1 MHz (prescaler 72 desde 72 MHz de reloj del sistema).
- Genera la señal STEP en PA8 mediante PWM de frecuencia variable.
- El período (ARR) se ajusta dinámicamente según velocidad requerida:  $ARR = (1000000 / freq\_hz) - 1$ .
- Duty cycle fijo al 50 % para pulsos simétricos.
- Rango operativo: 50 Hz a 20 kHz.

##### **TIM3 (General Purpose Timer) - Base de Tiempo del Sistema:**

- Configurado a 1 kHz (prescaler 7200, período 10), genera una interrupción cada 1 ms.
- Genera dos flags temporales no bloqueantes:
  - **flag\_1ms:** Dispara actualización de rampa S del motor (cada ciclo).
  - **flag\_100ms:** Dispara procesamiento ADC, PID térmico, telemetría UART y actualización LCD (contador interno: 100 ciclos).
- Actúa como reloj maestro que sincroniza todas las tareas del sistema.

#### 4.1.2. Adquisición de Temperatura (ADC + DMA)

El sensado térmico opera de forma completamente autónoma sin intervención del CPU:

- **ADC1:** Configurado en modo continuo para lectura del canal 0 (PA0 - divisor resistivo con NTC 100k $\Omega$ ).
- **DMA1\_Channel1:** Transfiere automáticamente cada conversión ADC a un buffer de 64 muestras (`adc_buffer[64]`).
- Cada 100 ms (`flag_100ms`), el firmware:
  1. Inicia nueva captura DMA de 64 conversiones.
  2. Calcula promedio aritmético del buffer para reducir ruido.
  3. Convierte el valor ADC promedio a temperatura mediante ecuación Beta.
  4. Aplica filtro IIR de primer orden ( $\alpha = 0,1$ ) para suavizado adicional.
- Esta estrategia reduce significativamente el ruido eléctrico sin sobrecargar el procesador.

#### 4.1.3. Bucle Principal (Superloop)

El `while(1)` ejecuta de forma iterativa las siguientes tareas:

1. **Refrescar Watchdog (IWDG):** Evita reset por timeout si el firmware se bloquea.
2. **Lectura de entradas:** Encoder, botones físicos, comandos UART entrantes.
3. **Gestión de alarmas:** Verificación continua del sistema de errores (prioridad máxima).
4. **Ejecución condicional según flags temporales:**
  - Si `flag_1ms`: Llamada a `motor_state_machine()`.
  - Si `flag_100ms`: Procesamiento ADC, cálculo PID, envío de telemetría.
5. **Gestión de menú o actualización de pantalla:** Según el modo activo.

### 4.2. Máquina de Estados Térmica

El control térmico está implementado como una FSM con 4 estados principales:

**Estados definidos:**

- **THERMAL\_IDLE:** Sistema en reposo, calefactor OFF. Espera comando de inicio (botón físico o UART).
- **THERMAL\_HEATING:** Calefactor activo, algoritmo PID regulando potencia.
  - Se activa un temporizador de seguridad (5 minutos). Si no se alcanzan 100°C en ese tiempo, se dispara `ERROR_HEATING_TIMEOUT`.
  - LED: Azul ( $T < \text{Target}-20^{\circ}\text{C}$ ) o Cyan ( $T > \text{Target}-20^{\circ}\text{C}$ ).

- **THERMAL\_READY**: Temperatura estabilizada ( $|T_{actual} - T_{target}| < 5^{\circ}\text{C}$ ). PID sigue activo para compensar perturbaciones. LED Verde.
- **THERMAL\_ERROR**: Fallo de seguridad detectado. Calefactor cortado, motor detenido, alarma visual/sonora. **Requiere reconocimiento manual** (presionar START) para salir.

**Transiciones:**

- IDLE  $\rightarrow$  HEATING: Botón Heater o comando UART H1.
- HEATING  $\rightarrow$  READY:  $|Error| < 5^{\circ}\text{C}$ .
- READY  $\rightarrow$  HEATING: Temperatura sale del rango por carga térmica (extrusión).
- Cualquier estado  $\rightarrow$  ERROR: Desconexión NTC, sobretemperatura ( $T > 275^{\circ}\text{C}$ ), timeout de calentamiento.
- ERROR  $\rightarrow$  IDLE: Solo mediante presión prolongada del botón START (reset intencional).

### 4.3. Máquina de Estados del Motor

La tracción mecánica también opera como FSM, con perfiles de aceleración suavizados mediante **curva S** (función coseno) para evitar sacudidas y vibraciones que afectarían la uniformidad del filamento.

**Estados definidos:**

- **MOTOR\_IDLE**: Motor deshabilitado, frecuencia = 0 Hz.
- **MOTOR\_ACCEL**: Aceleración gradual desde frecuencia mínima (50 Hz) hasta frecuencia objetivo. Duración: 2000 pasos de rampa ( $\sim 2$  segundos a 1 kHz).
- **MOTOR\_RUN**: Velocidad constante. Si cambia el setpoint de velocidad, se ajusta gradualmente (5 Hz por ciclo).
- **MOTOR\_DECEL**: Deceleración suave hasta frecuencia mínima, seguida de apagado. Duración: 2000 pasos de rampa.

**Protección Cold Protect**: El arranque del motor está bloqueado si  $T_{actual} < 100^{\circ}\text{C}$ , evitando obstrucciones mecánicas por filamento solidificado.

**Transiciones:**

- IDLE  $\rightarrow$  ACCEL: Botón START o comando UART R, solo si temperatura es segura.
- ACCEL  $\rightarrow$  RUN: Completados 2000 pasos de rampa.
- RUN  $\rightarrow$  DECEL: Botón STOP, comando UART S, o temperatura cae bajo umbral de seguridad.
- DECEL  $\rightarrow$  IDLE: Completada rampa de frenado.

## 4.4. Sistema de Seguridad Multicapa

### 4.4.1. Detección de Errores Críticos

El firmware verifica continuamente las siguientes condiciones de fallo:

1. **ERROR\_NTC\_DISCONNECTED**: Lectura ADC fuera de rango ( $\leq 100$  o  $\geq 4000$  cuentas digitales) o valor de temperatura = NaN (Not a Number). Indica cable suelto o sensor dañado.
2. **ERROR\_OVERTEMP**: Temperatura excede  $275^{\circ}\text{C}$ . Protección contra fuga térmica por fallo del PID o cortocircuito del MOSFET.
3. **ERROR\_HEATING\_TIMEOUT**: No se alcanzaron  $100^{\circ}\text{C}$  después de 5 minutos de calentamiento. Indica calefactor desconectado, contacto térmico deficiente o fallo en la etapa de potencia.

### 4.4.2. Enclavamiento de Seguridad (Safety Latch)

A diferencia de alarmas simples que auto-resetean al desaparecer la causa, este sistema implementa **latch de error**: una vez disparada la alarma, el sistema permanece bloqueado incluso si la condición se corrige (ej: reconectar el sensor). El usuario **debe presionar manualmente el botón START** para reconocer el evento y volver a modo IDLE. Esta filosofía evita reinicios automáticos peligrosos en equipos de alta temperatura.

### 4.4.3. Watchdog Independiente (IWDG)

El perro guardián interno (Independent Watchdog) del STM32 está configurado para resetear el microcontrolador si no se refresca cada 250 ms. Esto protege contra bloqueos del firmware por errores de software no previstos. El comando `HAL_IWDG_Refresh()` se ejecuta en cada iteración del loop principal.

## 4.5. Interacción de Módulos y Flujo de Ejecución

A continuación se describe el flujo típico de operación del sistema:

### Secuencia de arranque:

1. Usuario presiona botón HEATER (PB11) o envía H1 por UART.
2. Sistema transita a `THERMAL_HEATING`, activa PID y comienza calentamiento.
3. Cada 100 ms: ADC captura 64 muestras del termistor (DMA automático), firmware calcula temperatura mediante ecuación Beta y ejecuta PID. Salida PID determina si el pin PB10 está alto (calefactor ON) o bajo (OFF).
4. LED RGB indica progreso: Azul (lejano del target)  $\rightarrow$  Cyan (aproximándose)  $\rightarrow$  Verde (listo).
5. Al alcanzar temperatura, sistema transita a `THERMAL_READY`.

### Secuencia de extrusión:

1. Usuario presiona START (PA4). Firmware verifica temperatura segura ( $> 100^{\circ}\text{C}$ ).

2. Si es seguro: motor transita a MOTOR\_ACCEL. Timer 1 genera pulsos STEP en PA8 con frecuencia creciente según curva S.
3. Tras 2 segundos, motor alcanza velocidad objetivo (ej: 1500 Hz = 3 mm/s de filamento).
4. Bucle PID térmico compensa la pérdida de calor por flujo de material.
5. Al presionar STOP (PB12), motor decelera suavemente y se detiene.

#### Telemetría continua:

- Cada 100 ms, el firmware envía por UART un paquete CSV: timestamp, T\_actual, T\_target, PWM%, velocidad\_mm/s, error\_PID, codigo\_error, flags.
- La aplicación Python recibe estos datos, grafica en tiempo real y registra logs en archivos CSV para análisis post-proceso.

## 4.6. Diagrama de Flujo Operativo

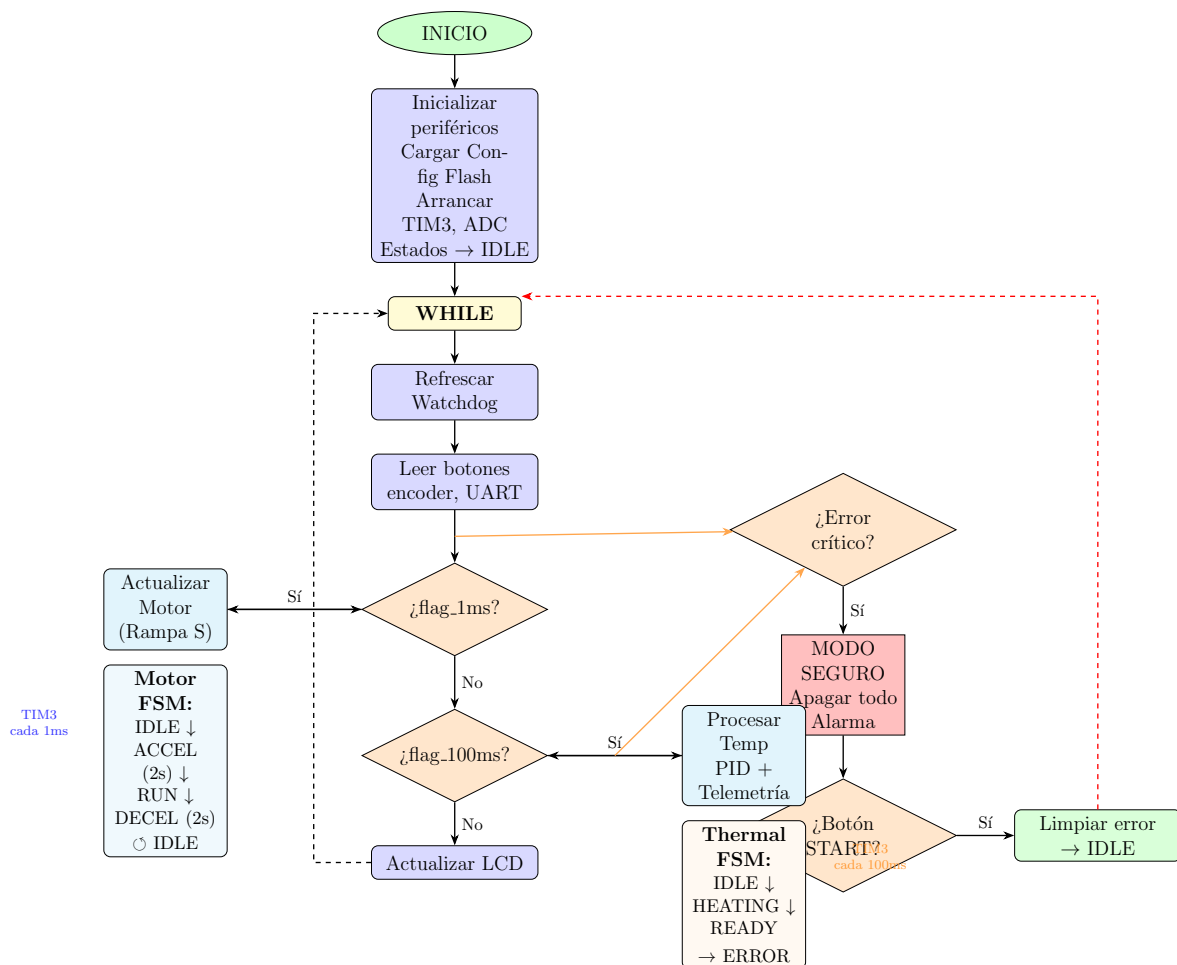


Figura 4: Diagrama de flujo del bucle principal y coordinación de máquinas de estado.

## **5. Programación**

### **5.1. Configuración de Periféricos**

### **5.2. Descripción del Firmware**

## **6. Etapas de Montaje y Ensayos Realizados**

## **7. Resultados y Especificaciones Finales**

Explicitar el grado de cumplimiento de las metas y objetivos planteados inicialmente.

- Consumo:
- Autonomía:
- Precisión:
- Velocidad:
- Rangos de trabajo:

## **8. Conclusiones**

Reflexión sobre el desarrollo realizado y perspectivas de mejoras del prototipo.

## **9. Referencias**

1. Hoja de datos STM32F103C8T6.
2. Manual de referencia RM0008 STM32F101xx, STM32F102xx, STM32F103xx.

## **A. Anexos**